

Muddiest Points II

1. Faktoren in R: Verständnis und Anwendung

Frage: Was ist ein Faktor und wann sollte eine Variable in einen Faktor umgewandelt werden?

Antwort: Ein Faktor ist ein Datentyp für kategoriale Variablen (z.B. Experimentalgruppen, Geschlecht, Studienfach).

In R wird ein Faktor dann verwendet, wenn kategoriale Information abgebildet werden soll, also z.B.:

- Gruppenzugehörigkeit (Kontrollgruppe vs. Interventionsgruppe)
- Klassifizierungen (low, medium, high)
- Messzeitpunkt (t1, t2, t3, t4)

Der entscheidende Punkt ist, dass Faktoren diskrete Ausprägungen (Levels) haben und keine mathematische Bedeutung. Das Umwandeln in einen Faktor ist wichtig, weil viele Funktionen in R (insbesondere statistische Verfahren wie z.B. eine ANOVA) erkennen müssen, dass es sich um eine kategoriale Variable handelt. Wenn eine Variable nicht als Faktor vorliegt, kann R sie falsch interpretieren (z.B. als numerische Variable), was zu fehlerhaften Analysen führen kann.

Frage: Warum ist es wichtig, Variablen korrekt als Faktoren zu definieren?

Antwort: Weil viele Funktionen in R den Datentyp nutzen, um zu entscheiden, wie eine Variable behandelt wird. Der Datentyp steuert in R das Verhalten von Funktionen:

- Statistische Modelle unterscheiden zwischen **metrischen** und **kategorialen** Variablen
- Visualisierungen (z.B. in ggplot) verwenden Faktoren für Gruppierungen
- Auswertungen (z.B. Mittelwertsvergleiche) setzen, dass Gruppierungen als Faktoren definiert sind

So muss beispielsweise bei einer ANOVA die Gruppenvariable ein Faktor sein, damit R weiss, dass diese Variable die Gruppen definiert, die verglichen werden sollen. Ohne Faktor würde R die Variable ggf. als Zahl behandeln und Unterschiede falsch interpretieren.

Frage: Was ist der Unterschied zwischen `factor()`, `as.factor()` und `as_factor()`?

Antwort: Die drei Funktionen machen im Kern dasselbe: Sie wandeln eine Variable in einen Faktor um.

Der Unterschied liegt darin, wie viel Kontrolle besteht und wie die Faktoren-Levels bestimmt werden.

as.factor() – die einfache Standardlösung

- Teil von Base R
 - Wandelt eine Variable direkt in einen Faktor um
 - Die Levels werden automatisch alphabetisch sortiert
- ➔ Kann verwendet werden, wenn die Reihenfolge egal ist und einfach schnell ein Faktor benötigt wird

factor() – volle Kontrolle

- Ebenfalls Teil von Base R
- Es kann selbst festgelegt werden, welche Levels existieren und in welcher Reihenfolge diese stehen

Beispiel zur Demonstration:

```
# Beispiel-Datensatz
zufriedenheit <- c("mittel", "hoch", "niedrig", "hoch", "mittel")
```

```
# Standard: automatische (alphabetische) Reihenfolge
zufriedenheit_factor_auto <- as.factor(zufriedenheit)
levels(zufriedenheit_factor_auto)
# Ergebnis: "hoch" "mittel" "niedrig"
```

```
# Mit factor(): Reihenfolge bewusst festlegen
zufriedenheit_factor_geordnet <- factor(
  zufriedenheit,
  levels = c("niedrig", "mittel", "hoch"),
  ordered = TRUE
)
```

```
levels(zufriedenheit_factor_geordnet)
# Ergebnis: "niedrig" "mittel" "hoch"
```

- ➔ Kann verwendet werden, wenn die Reihenfolge wichtig ist und Kategorien bewusst definiert werden sollen

as_factor() – praktisch bei importierten Daten

- Teil des Tidyverse (forcats)
- Im Unterschied zu as.factor() übernimmt die Funktion vorhandene Codierungen (z.B. aus SPSS)

- Auch basieren die Levels hier auf der Reihenfolge der Daten im Datensatz, nicht auf dem Alphabet
- ➔ Kann verwendet werden, wenn Daten importiert wurden (z.B. aus SPSS) und die ursprüngliche Reihenfolge beibehalten werden soll

Tipp: Über die Syntax `?Funktionsname()` kann die Dokumentation einer Funktion aufgerufen werden. Dort ist beschrieben, welche Argumente die Funktion erwartet, welche Default-Einstellungen gelten und welche Ausgabe erzeugt wird. Hierüber lassen sich auch Unterschiede zwischen ähnlichen Funktionen feststellen.

2. Umgang mit fehlenden Werten

Frage: *Beim Umgang mit fehlenden Werten ist mir noch nicht ganz klar, wie man entscheidet, ob fehlende Werte einfach ausgeschlossen, ersetzt oder genauer analysiert werden sollten. Ich würde gerne besser verstehen, welche Auswirkungen das hat.*

Antwort: Der Umgang mit fehlenden Werten hängt immer davon ab, **warum Werte fehlen** und welche Fragestellung verfolgt wird. Erst wenn verstanden ist, ob fehlende Werte zufällig oder systematisch auftreten, kann entschieden werden, ob sie entfernt, ersetzt oder genauer untersucht werden sollten.

Der zentrale Punkt beim Umgang mit fehlenden Werten ist, dass sie nicht einfach nur ein technisches Problem darstellen, sondern ein inhaltliches und statistisches Problem. Fehlende Werte können nämlich die Ergebnisse einer Analyse verzerren; insbesondere dann, wenn sie nicht zufällig entstehen. Deshalb sollte man fehlende Werte immer in einem ersten Schritt **analysieren und verstehen**, bevor man sie behandelt. In der Statistik wird dabei zwischen drei grundlegenden Formen unterschieden:

- Wenn Werte komplett zufällig fehlen (**MCAR**), besteht kein Zusammenhang mit anderen Variablen oder den fehlenden Werten selbst. Ein Beispiel wäre ein technischer Ausfall eines Messgeräts. In diesem Fall ist es meist unproblematisch, Fälle mit fehlenden Werten zu entfernen, da keine systematische Verzerrung zu erwarten ist.
- Anders ist die Situation, wenn fehlende Werte mit anderen beobachteten Variablen zusammenhängen (**MAR**). Beispielsweise könnte es sein, dass jüngere Personen eine bestimmte Frage häufiger überspringen. Hier ist das Fehlen bereits strukturiert, auch wenn es nicht direkt vom fehlenden Wert selbst abhängt. Werden solche Fälle einfach entfernt, kann das dazu führen, dass bestimmte Gruppen im Datensatz unterrepräsentiert sind.
- Besonders kritisch ist der Fall, dass fehlende Werte direkt mit dem fehlenden Wert selbst zusammenhängen (**MNAR**). Ein klassisches Beispiel wäre, dass Personen mit hoher Ausprägung eines Merkmals (z.B. Angst) eine Studie abbrechen. In diesem

Fälle gehen systematisch genau die Werte verloren, die für die Analyse besonders relevant wären. Ein einfaches Entfernen dieser Fälle würde die Ergebnisse stark verzerren.

Vor diesem Hintergrund wird deutlich, dass die Entscheidung über den Umgang mit fehlenden Werten keine rein technische ist, sondern immer von der Datenstruktur und der Forschungsfrage abhängt.

Wenn fehlende Werte nur in geringem Umfang auftreten und plausibel zufällig sind, kann es sinnvoll sein, diese Fälle zu entfernen. Dies ist die einfachste Vorgehensweise und wird von vielen statistischen Verfahren direkt umgesetzt. Allerdings geht dabei immer ein Teil der Daten verloren, was die Aussagekraft der Analyse verringern kann.

Eine alternative Strategie besteht darin, fehlende Werte zu ersetzen (Imputation). Dabei werden fehlende Werte durch geschätzte Werte ergänzt, beispielsweise durch den Mittelwert oder komplexere Verfahren wie Regression oder Multiple Imputation. Der Vorteil liegt darin, dass der Datensatz vollständig bleibt. Allerdings basiert diese Vorgehensweise immer auf Annahmen, und insbesondere einfache Methoden können die Varianz unterschätzen und Zusammenhänge verzerren.

In vielen Fällen ist es zudem sinnvoll, fehlende Werte nicht nur zu behandeln, sondern aktiv zu analysieren. Das bedeutet, zu prüfen, ob systematische Muster vorliegen und ob das Fehlen selbst bereits eine relevante Information darstellt.

- ➔ Zusammenfassend lässt sich sagen, dass der Umgang mit fehlenden Werten immer einem dreistufigen Verfahren folgt: Zuerst müssen fehlende Werte identifiziert werden, anschliessend sollte ihre Entstehung bestmöglich verstanden werden, und erst darauf aufbauend kann eine geeignete Behandlungsstrategie gewählt werden. Der entscheidende Schritt ist dabei das Verständnis der Ursache, da hiervon abhängt, ob eine Analyse unverzerrt möglich ist.
- ➔ Idealerweise macht man sich schon vor einer Datenerhebung Gedanken darüber wie fehlende Werte entstehen könnten und beschreibt schon in einer Präregistrierung wie mit diesen umgegangen wird.

3. Wide vs. Long: Wann wird welches Format benötigt?

Frage: *Bei der Umformung von Wide- zu Long-Format verstehe ich grundsätzlich den Code, aber ich bin noch unsicher, wann ein Long-Datensatz für eine Analyse wirklich notwendig ist und woran ich erkenne, was wann besser geeignet ist.*

Antwort: Ob ein Datensatz im Wide- oder Long-Format vorliegen sollte, hängt davon ab, wie die Daten analysiert werden sollen. Insbesondere dann, wenn mehrere Messungen pro

Person vorliegen (und diese in einer Analyse verglichen werden sollen), ist das Long-Format meist notwendig, da viele statistische Verfahren und Funktionen in R davon ausgehen, dass jede Zeile eine einzelne Beobachtung darstellt.

Der Unterschied zwischen Wide- und Long-Format betrifft zunächst nur die Struktur der Daten, nicht ihren Inhalt. Beide Formate enthalten die gleichen Informationen, organisieren diese jedoch auf unterschiedliche Weise. Im Wide-Format entspricht eine Zeile typischerweise einer Person, während verschiedene Messungen (etwa zu unterschiedlichen Zeitpunkten) in separaten Spalten gespeichert sind. Dieses Format ist oft intuitiv und übersichtlich, insbesondere wenn man sich einen ersten Eindruck vom Datensatz verschaffen möchte.

Im Long-Format wird diese Struktur gewissermassen «aufgelöst». Hier entspricht jede Zeile einer einzelnen Messung, und zusätzliche Variablen geben an, zu welcher Person und zu welchem Messzeitpunkt diese Beobachtung gehört. Dadurch entsteht eine Datenstruktur, in der jede Zeile genau eine Beobachtung repräsentiert.

Entscheidend ist nun, dass viele Auswertungsmethoden nicht mit «nebeneinanderliegenden» Within-Subjects Messungen arbeiten können, sondern darauf angewiesen sind, dass Messungen untereinander organisiert sind. Im Wide-Format sind diese Messungen auf mehrere Spalten verteilt, was es erschwert, systematisch über diese Dimensionen hinweg zu analysieren. Im Long-Format hingegen kann beispielsweise eine Variable «Messzeitpunkt» direkt in Analysen einbezogen werden, wodurch sich Veränderungen über die Zeit oder Unterschiede zwischen Bedingungen deutlich einfacher untersuchen lassen.

Vor diesem Hintergrund lässt sich die Entscheidung für ein bestimmtes Datenformat gut über die Art der Variablen treffen, die in der Analyse verwendet werden. Eine hilfreiche Unterscheidung ist dabei die zwischen **Between- und Within-Variablen**.

Wenn in einer Analyse ausschliesslich **Between-Variablen** vorkommen, also jede Person genau eine Ausprägung pro Variable hat (z.B. Geschlecht, Gruppe), kann der Datensatz problemlos im **Wide-Format** vorliegen. In diesem Fall entspricht jede Zeile einer Person, und es gibt keine mehrfachen Messungen pro Einheit.

Sobald jedoch mindestens eine **Within-Variable** vorliegt, also mehrere Messungen pro Person (etwa über verschiedene Zeitpunkte, Bedingungen oder Items hinweg), ist in der Regel das **Long-Format** notwendig. Der Grund ist, dass solche Analysen davon ausgehen, dass jede einzelne Messung als eigene Beobachtung vorliegt. Im Long-Format können diese Messungen dann über eine zusätzliche Variable (z.B. «Zeitpunkt» oder «Bedingung») systematisch analysiert werden.

➔ In der praktischen Arbeit ergibt sich daraus häufig folgendes Vorgehen: Daten liegen oft zunächst im Wide-Format vor, da sie so einfacher erhoben oder gespeichert

werden können. Für Messwiederholungs-Analysen werden sie dann jedoch ins Long-Format überführt, da dieses besser mit den Anforderungen statistischer Verfahren und Datenmanipulation in R übereinstimmt. Je nach Analysemethode, wird dann der Wide (between-subjects Analysen) oder Long Datensatz (within-subjects Analysen) verwendet.

4. Reihenfolge der Datenaufbereitung: Was kommt wann?

Frage: Allgemein fällt es mir noch etwas schwer, die einzelnen Schritte der Datenaufbereitung in eine sinnvolle Reihenfolge zu bringen, also z.B. wann man zuerst filtert, Variablen berechnet, fehlende Werte prüft oder Daten umformt.

Antwort: Eine sinnvolle Datenaufbereitung folgt meistens der Logik:

- Daten importieren;
- Überblick gewinnen;
- Daten bereinigen;
- fehlende Werte prüfen;
- neue Variablen berechnen;
- Daten ggf. umformen (z.B. Wide und Long Format);
- Datensätze final prüfen;
- Datenqualität analysieren;
- Daten visualisieren;
- Daten analysieren.

Die genaue Reihenfolge hängt zwar vom Datensatz und der Fragestellung ab, aber wichtig ist: Man sollte nicht sofort mit Analysen oder komplexen Transformationen beginnen, sondern zuerst sicherstellen, dass man den Datensatz versteht und dass die Variablen korrekt vorliegen.

Datenaufbereitung ist kein einzelner Schritt, sondern ein Prozess! Ziel ist es, aus Rohdaten einen Datensatz zu erzeugen, der für die geplante Analyse geeignet, nachvollziehbar und möglichst fehlerfrei ist.

Am Anfang steht immer der **Datenimport**. Hier werden die Rohdaten in R eingelesen. Direkt nach dem Import sollte man den Datensatz nicht sofort verändern, sondern sich zunächst einen Überblick verschaffen: Welche Variablen gibt es? Wie viele Fälle enthält der Datensatz? Welche Datentypen haben die Variablen? Gibt es auffällige Werte, leere Spalten oder unerwartete Kodierungen? Dieser erste Überblick ist wichtig, weil spätere Schritte sonst auf falschen Annahmen beruhen können.

Danach folgt meist eine erste **technische Bereinigung**. Dazu gehört bspw. das Vereinheitlichen von Variablennamen, die Behebung offensichtlicher Importprobleme oder

die Entfernung nicht benötigter Hilfsspalten. Auch das Umkodieren versteckter fehlender Werte gehört hierher: Wurden fehlende Werte etwa als 999, -999 oder leere Strings gespeichert, sollten sie möglichst früh in echte NA-Werte umgewandelt werden.

Abschliessend sollten **fehlende Werte** systematisch geprüft werden. Dabei geht es zunächst darum, wo Werte fehlen: insgesamt, pro Variable oder vielleicht nur bei bestimmten Gruppen. Danach sollte überlegt werden, warum diese Werte fehlen könnten. Dieser Schritt sollte relativ früh erfolgen, weil fehlende Werte viele spätere Entscheidungen beeinflussen.

Erst wenn der Datensatz grundsätzlich verstanden und bereinigt ist, werden Variablen in **passende Datentypen** überführt. Numerische Variablen sollten tatsächlich numerisch sein, kategoriale Variablen sollten häufig als Faktoren vorliegen, und Zeichenvariablen als solche erkannt werden. Dieser Schritt ist wichtig, weil viele Funktionen in R unterschiedlich reagieren, je nachdem, ob eine Variable numerisch, kategorial oder als Text gespeichert ist.

Danach folgt häufig die **inhaltliche Umkodierung oder Berechnung neuer Variablen**. Hier werden z.B. Summenwerte, Mittelwerte, Differenzwerte, Gruppenvariablen oder z-standardisierte Variablen erstellt. Dieser Schritt sollte erst erfolgen, wenn klar ist, dass die zugrunde liegenden Variablen korrekt bereinigt und kodiert sind. Sonst berechnet man neue Variablen auf Basis fehlerhafter Ausgangswerte.

Das **Filtern von Fällen** kann an unterschiedlichen Stellen sinnvoll sein. Offensichtlich unbrauchbare Fälle oder Testpersonen, die gar nicht zur Zielgruppe gehören, können früh ausgeschlossen werden. Analysespezifische Filter (z.B. nur vollständige Fälle für eine bestimmte Auswertung) sollten dagegen eher später und bewusst gesetzt werden. Wichtig ist dabei, Filter immer transparent zu dokumentieren, weil sie direkt beeinflussen, welche Personen oder Beobachtungen in die Analyse eingehen.

Die **Umformung von Wide zu Long** (oder umgekehrt) erfolgt meist dann, wenn klar ist, welche Variablen analysiert werden sollen und welche Datenstruktur die Analyse benötigt. Wenn bspw. mehrere Messzeitpunkte pro Person analysiert werden sollen, wird der Datensatz ins Long-Format gebracht. Diese Umformung sollte nicht ganz am Anfang «blind» erfolgen, sondern dann, wenn man verstanden hat, welche Spalten zusammengehören und welche Information durch die Umformung neu als Variable abgebildet werden soll.

Ganz am Ende steht die **abschliessende Plausibilitätsprüfung**. Man prüft also noch einmal: Stimmen die Fallzahlen? Sind die Variablen korrekt kodiert? Gibt es unerwartete fehlende Werte? Haben neu berechnete Variablen plausible Wertebereiche? Sieht der Datensatz so aus, wie er für die Analyse aussehen soll? Erst danach sollte mit den eigentlichen statistischen Analysen begonnen werden.

5. Daten einlesen in R

Frage: Zum Teil ist das Einlesen immer mal ein Mysterium. Ab und zu funktioniert das Einlesen via Code, ab und zu nicht. Manchmal funktioniert auch Code, der in einer Übung noch funktioniert hat, eine Woche später plötzlich nicht mehr – ohne dass bewusst etwas geändert wurde. Wie kann man sicherstellen, dass das Einlesen zuverlässig funktioniert, auch bei Replikation durch andere?

Antwort: Probleme beim Einlesen von Daten hängen fast immer mit Dateipfaden und Arbeitsverzeichnissen zusammen. Der wichtigste Grundsatz für reproduzierbares Arbeiten ist daher: Arbeite immer innerhalb eines R-Projekts, verwende relative Pfade und öffne deine Skripte aus dem Projekt heraus.

Dass also das Einlesen von Daten in R manchmal funktioniert und manchmal nicht, liegt in den meisten Fällen nicht am eigentlichen Code (z.B. `read_csv()`), sondern daran, dass R nicht mehr «weiss», wo sich die Datei befindet. Der zentrale Begriff hierbei ist das Working Directory, welches angibt, von welchem Ordner aus R relative Dateipfade interpretiert.

Wenn eine Datei mit einem Befehl wie `read_csv("data/raw/data.csv")` eingelesen wird, geht R davon aus, dass es sich aktuell im richtigen Ausgangsordner befindet. Ist das Working Directory jedoch ein anderes, etwa weil das Skript aus einem anderen Ordner geöffnet wurde oder weil sich die Ordnerstruktur verändert hat, kann R die Datei nicht mehr finden, obwohl sich weder der Code noch die Datei selbst geändert haben.

Um dieses Problem zu vermeiden, ist es zentral, mit R-Projekten zu arbeiten. Wenn ein Projekt verwendet und das Skript aus diesem Projekt heraus geöffnet wird, wird automatisch das Projektverzeichnis als Working Directory gesetzt. Dadurch bleiben alle relativen Pfade innerhalb dieses Projekts stabil und funktionieren unabhängig davon, auf welchem Computer oder zu welchem Zeitpunkt das Skript ausgeführt wird.

Ein weiterer wichtiger Punkt ist die Verwendung von relativen statt absoluten Pfaden. Solange die Projektstruktur gleichbleibt, funktionieren relative Pfade zuverlässig; dies unabhängig vom Gerät oder Speicherort.

Darüber hinaus ist es wichtig, dass der Code zum Einlesen der Daten im Skript selbst enthalten ist und nicht nur interaktiv in der Konsole ausgeführt wurde. Nur so kann eine andere Person (oder man selbst zu einem späteren Zeitpunkt) den gesamten Workflow nachvollziehen und reproduzieren.

Komm auf uns zu, falls das Einlesen mal wieder nicht klappt – vielleicht können wir das Mysterium beseitigen 😊

6. YAML-Header in Quarto

Frage: Welche Basic-Einstellungen gehören in den YAML-Header in einem Quarto-Skript?

Antwort: Der YAML-Header steht ganz oben im Quarto-Dokument und legt fest, wie das Dokument gerendert werden soll. Minimal braucht man vor allem Angaben wie `title` und `format`. Zusätzlich sind Angaben wie `author`, `date` und globale Ausführungsoptionen unter `execute` sehr hilfreich.

Die Angaben im YAML-Header steuern nicht den Inhalt selbst, sondern die Art, wie Quarto das Dokument verarbeitet und ausgibt. Besonders wichtig ist dabei die Option **format**, weil sie bestimmt, ob aus dem Quarto-Skript z.B. eine HTML-Datei, ein Word-Dokument oder ein PDF erstellt wird. Für das korrekte Rendern ist also vor allem entscheidend, dass Quarto weiss, welches Ausgabeformat erzeugt werden soll.

Neben dem Format sind **title**, **author** und **date** typische Basisangaben. Sie sind nicht immer technisch zwingend notwendig, aber sehr hilfreich, weil sie im gerenderten Dokument sichtbar machen, worum es geht und von wem das Dokument stammt. Dies stärkt auch die Reproduzierbarkeit!

Wenn globale Einstellungen für alle Code-Chunks gelten sollen, verwendet man im YAML-Header den Bereich **execute**. Dort kann man festlegen, ob Code, Warnmeldungen oder Ausgaben im gerenderten Dokument erscheinen sollen. Das ist praktisch, weil man solche Optionen dann nicht in jedem einzelnen Code-Chunk wiederholen muss. Quarto dokumentiert diese Ausführungsoptionen als globale Optionen im YAML-Header; einzelne Code-Chunks können sie bei Bedarf trotzdem überschreiben.

Darüber hinaus gibt es sehr viele weitere optionale Einstellungen, z.B. für Inhaltsverzeichnisse, nummerierte Überschriften, Themes, Code-Folding, Tabellen, Abbildungen oder Layout. Diese Optionen sind nützlich, aber nicht alle für den Einstieg nötig. Für den Anfang reicht es völlig, den YAML-Header als Ort für die zentralen Dokumenteinstellungen zu verstehen: Titel, Autor:in, Datum, Ausgabeformat und globale Code-Optionen.

7. Arbeiten mit Code: Wie finde ich die richtigen Befehle?

Frage: Ich muss ehrlich sagen, dass ich vieles mit KI lösen muss, da ich mir die Codes selbst nicht herleiten kann. Vielleicht könnten wir deshalb nochmals anschauen, welche Überlegungen ich machen muss, um mir die richtigen Codes rauszusuchen (z.B. aus einem Cheatsheet) und wie ich weiss, welche Argumente jeweils in die Klammern kommen.

Antwort: Du musst Code nicht auswendig können. Entscheidend ist, dass du verstehst, welches Problem du lösen willst, und dann gezielt passende Funktionen auswählst und deren Argumente nachschaust. Dafür gibt es in R zu jeder Funktion eine eigene Hilfeseite mit Erklärungen und Beispielen.

Es ist völlig normal, und auch für erfahrene Programmierer:innen üblich, Code nicht komplett auswendig zu können. Entscheidend ist nicht das Auswendiglernen einzelner Befehle, sondern das Verständnis für den **Workflow**: Was ist mein Ziel, welche Art von Operation brauche ich, und welches Werkzeug in R kann das leisten?

Ein sinnvoller erster Schritt ist daher immer, das Problem inhaltlich zu klären. Sobald klar ist, welche Art von Aufgabe vorliegt, kann man gezielt im [Cheatsheet](#) oder in den Folien nach passenden Funktionen suchen. Die Cheatsheets sind genau dafür gedacht: Sie geben einen strukturierten Überblick über typische Befehle für häufige Ausgaben.

Wenn man eine passende Funktion gefunden hat, stellt sich oft die Frage, welche Argumente benötigt werden. Hier ist die wichtigste Ressource die **Hilfeseite in R** selbst. Diese kann mit `?funktionsname` oder `help(funktionsname)` aufgerufen werden. Dort ist genau dokumentiert, welche Argumente die Funktion erwartet, welche Standardwerte (Defaults) gelten, was die Funktion zurückgibt und wie sie anhand konkreter Beispiele korrekt verwendet wird. Diese Beispiele sind besonders hilfreich, weil sie zeigen, wie die Funktion in der Praxis eingesetzt wird. Es ist völlig legitim und sinnvoll, sich daran zu orientieren und sie auf die eigene Aufgabe anzupassen.

Wichtig ist auch zu verstehen, dass in den Übungen und Hausaufgaben kein «verstecktes Wissen» erwartet wird. Es wird nur Code verlangt, der in der Vorlesung behandelt wurde und in den Folien nachgelesen werden kann. Die Übungen dienen genau dazu, diese Befehle einzuüben und ein Gefühl dafür zu entwickeln, wann welcher Schritt sinnvoll ist.

Dabei ist es auch wichtig zu betonen, dass es in R oft mehrere Wege zum Ziel gibt. Unterschiedliche Lösungen sind in vielen Fällen möglich und teilweise auch eine Frage des persönlichen Stils. Für die Übungen ist jedoch entscheidend, dass eine nachvollziehbare und korrekte Lösung verwendet wird; nicht, dass alle möglichen Varianten bekannt sind.

Die Verwendung von LLMs für Programmier-Aufgaben ist heutzutage normal und auch okay. Wichtig ist, dass du KI-generierten Code nachvollziehen und verifizieren kannst. Dafür empfehlen sich «Doppelchecks» mit den Cheatsheets und/oder der Hilfeseite in R.

8. Mit welchem Datensatz soll ich arbeiten?

Frage: *Eigentlich fand ich die Inhalte sehr klar. Für mich ist bei den Aufgaben noch nicht immer klar, wann man mit welchem Datensatz weiterarbeiten muss (z.B. in der aktuellen Hausübung) oder wie man das aus der Aufgabe herauslesen kann.*

Antwort: In den Übungen und Hausübungen ist immer angegeben, welcher Datensatz verwendet werden soll (aber wir werden uns bemühen das noch deutlicher zu machen!). Häufig arbeitet ihr mit dem Datensatz `dat_full`, der im Verlauf des Semesters schrittweise erweitert wird. Wenn ein anderer Datensatz benötigt wird, wird das jeweils erwähnt und zusätzlich über einen Download-Button bereitgestellt (wie z.B. bei der letzten R Hausübung).

9. Pipe-Operatoren in R

Frage: *Was ist der Unterschied zwischen den zwei Pipe-Zeichen?*

Antwort: Die beiden Pipe-Operatoren `|>` (Base R) und `%>%` (Tidyverse) funktionieren in den meisten Fällen gleich: Sie leiten das Ergebnis eines Befehls in den nächsten weiter. Für das Seminar ist der Unterschied nicht entscheidend; empfohlen wird `|>`, da es Teil von Base R ist und immer verfügbar ist.

Für Informationen zur Unterscheidung der Pipe Operatoren, siehe <https://r4ds.hadley.nz/data-transform.html#sec-the-pipe>, Kapitel 3.4.

10. Pfade und `here::here()`

Frage: *Wann benutzt man beim Speichern eines Datensatzes `here::here()`?*

Antwort: `here::here()` kann immer dann verwendet werden, wenn Dateien innerhalb eines Projekts gespeichert oder geladen werden sollen. Es ist eine einfache Möglichkeit, **relative Pfade** zu erstellen, die unabhängig davon funktionieren, von wo aus das Skript ausgeführt wird.

Beim Speichern oder Einlesen von Dateien in R ist nicht nur der Befehl selbst wichtig (z.B. `write_csv()` oder `read_csv()`), sondern vor allem der **Dateipfad**. Genau hier entstehen viele Probleme, insbesondere wenn Skripte später erneut ausgeführt oder von anderen Personen verwendet werden. Die Funktion `here::here()` hilft dabei, solche Probleme zu vermeiden, indem sie Pfade relativ zum Projektordner erstellt.

- ➔ Damit ist `here::here()` eine von mehreren Möglichkeiten, relative Pfade zu setzen; also Pfade, die nicht vom konkreten Speicherort auf dem Computer abhängen, sondern von der Struktur des Projekts.

Ein Beispiel: `write_csv(dat, here::here("data", "processed", "dat_clean.csv"))`

Hier wird die Datei im Ordner `data/processed/` innerhalb des Projekts gespeichert; unabhängig davon, auf welchem Computer oder an welchem Speicherort das Projekt liegt.

Der grosse Vorteil ist die Reproduzierbarkeit: Solange die Ordnerstruktur gleichbleibt und das Skript aus dem Projekt heraus geöffnet wird, funktioniert dieser Pfad immer zuverlässig. Wichtig dabei ist, dass `here::here()` nur dann korrekt funktioniert, wenn das Skript aus dem R-Projekt heraus geöffnet wird! Nur dann erkennt R automatisch den Projektordner als Ausgangspunkt aller Pfade.

11. Änderungen am Datensatz: Speichern und dokumentieren

Frage: *In den Übungen nehmen wir häufig Veränderungen am Datensatz `dat_full` oder an einzelnen Variablen vor, ohne diese anschliessend abzuspeichern. Daher sind wir uns unsicher, wie der aktuelle Datensatz genau aussehen soll. Wann müssen Änderungen gespeichert werden und welche Anpassungen gehören ins Codebook?*

Antwort: `dat_full` sollte aktuell 159 Zeilen und 37 Variablen haben. Wir haben eine Variable ergänzt, nämlich `mmq_mean`. Ausserdem haben wir 5 Variablen mit `*10` umgewandelt – `pre1`, `pre2`, `pre3`, `pre4` und `post`. Idealerweise habt ihr dann noch den Namen der Variable «`csvtm_sum`» korrigiert. Und genau diese Anpassungen solltet ihr auch im Codebook vornehmen.

12. Fragen zur Abschlussabgabe

Frage: *Wann erhalten wir etwa den finalen Datensatz, welchen wir dann für die Abschlussarbeit benötigen?*

Antwort: Wir planen euch die individuellen, simulierten Datensätze mit EH 12 (also 13.05.) zu geben.

Frage: *Bekommen wir für die Schlussabgabe einen neuen Datensatz, bei dem wir andere Sachen umbenennen müssen und es Fehlerwerte drin hat?*

Antwort: Ja und ja. Wir werden in EH 12 näher darüber sprechen und ihr bekommt von uns eine Checkliste, der Dinge die ihr in euren individuellen, simulierten Datensätze überprüfen müsst.

Frage: *Bis wann kann man eine Fristverlängerung einreichen für die Abgabe der Abschlussabgabe?*

Antwort:

Bitte komm mit dieser Frage persönlich auf uns zu.